Consider the following example that has the Document Type Definition (DTD) embedded in the XML:

```
<?xml version="1.0"?>
<!DOCTYPE note [
<!ELEMENT note (to,from,header,heading,body)>
<!ELEMENT to (#PCDATA)>
<!ELEMENT from (#PCDATA)>
<!ELEMENT heading (#PCDATA)>
<!ELEMENT body (#PCDATA)>
]>
<note2>
<to>Tove</to>
<from>Jani</from>
<heading>Reminder</heading>
<body>Don't forget me this weekend</body>
</note>
```

An incorrect element *note2* in the XML is detected by the parser, and an error is thrown by the *xml-parse-file* function as shown below:

```
(xml-parse-file "/tmp/error.xml" t)
(error "XML: (Not Well-Formed) Invalid end tag 'note'
(expecting 'note2')")
```

There are a number of built-in functions in the library. The 'xml-parse-elem-type' function will parse the Document Type Definition and will return a list as illustrated below:

```
(xml-parse-elem-type string) ;; Syntax

(xml-parse-elem-type "<!DOCTYPE note [
<!ELEMENT note (to,from,header,body)>
<!ELEMENT to (#PCDATA)>
<!ELEMENT from (#PCDATA)>
<!ELEMENT heading (#PCDATA)>
<!ELEMENT body (#PCDATA)>
]>")

(seq "to" "from" "header" "body")
```

The numeric entities can be replaced with their respective characters using the *xml-substitute-numeric-entities* function as shown below:

```
(xml-substitute-numeric-entities string) ;; Syntax

(xml-substitute-numeric-entities "<!ENTITY ora &#34;O'Reilly
&amp; Associates&#34;>")

"<!ENTITY ora \"O'Reilly &amp; Associates\">"
```

The *xml-escape-string* function, on the other hand, escapes the string literals with valid XML character data. For example:

```
(xml-escape-string string) ;; Syntax

(xml-escape-string "ENTITY ora O'Reilly & Associates")
"ENTITY ora O&apos;Reilly &amp; Associates"
```

The actual source code of the *xml-escape-string* function is given below for reference:

```
(defun xml-escape-string (string)
  (with-temp-buffer
    (insert string)
(dolist (substitution '(("&" . "&amp;")
                        ("<" . "&lt;")
                       (">" . "&gt;")
                       ("'" . "&apos;")
                       ("\"" . "&quot;")))
      (goto-char (point-min))
      (while (search-forward (car substitution) nil t)
        (replace-match (cdr substitution) t t nil)))
    (buffer-string)))
```

The *xml-substitute-special* function returns a string after substituting entity and character references. An example is given below:

```
(xml-substitute-special string) ;; Syntax
(xml-substitute-special "<!ENTITY ora 'O'Reilly &amp;
Associates'>")
"<!ENTITY ora 'O'Reilly & Associates'>"
```

A number of other internal Emacs Lisp functions are available in the *xml.el* file: *xml–entity-replacement-text*, *xml-parse-string*, *xml-parse-dtd*, *xml-maybe-do-ns*, *xml-parse-attlist*, *xml-skip-dtd*, and *xml-remove-comments*.

The default list of XML name spaces is defined in the *xml-default-ns* constant as shown below:

```
(defconst xml-default-ns '(("" . "")
     ("xml" . "http://www.w3.org/XML/1998/namespace")
     ("xmlns" . "http://www.w3.org/2000/xmlns/"))
 "Alist mapping default XML namespaces to their URIs.")
```

You are encouraged to read the *xml.el* source code to study the implementation of the different functions it provides. END

By: Shakthi Kannan

The author is a free software enthusiast who blogs at *shaktimaan.com*.